“Multi-Process Command Pipeline System: Concurrent Linux Command Execution Using Pipes”

A Project Report submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

By

BATHULA NEHA	**2520090013**
CHAKRIKA REDDY	**2520090044**
HARSHIKA	**2520090047**

Under the Esteemed Guidance of

Dr. K. Anusha
Assistant Professor
Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled **"Multi-Process Command Pipeline System"** is a record of Bonafide work of **BATHULA NEHA - 2520090013, CHAKRIKA REDDY - 2520090044, HARSHIKA - 2520090047** submitted in
partial fulfillment for the award of B. Tech in Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

BATHULA NEHA	**2520090013**
CHAKRIKA REDDY	**2520090044**
HARSHIKA	**2520090047**

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is to certify that the mini project based report entitled **"Multi-Process Command Pipeline System"** is a bonafide work done and submitted by **Bathula Neha – 2520090013, Chakrika Reddy – 2520090044, Harshika – 2520090047**,

in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science and Information Technology, K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Guide

Signature of the Course Coordinator **Signature of the HOD**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to our Course Coordinator Dr. K. Anusha, and our guide Dr. K. Anusha, Department of Computer Science, for her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P Venkateshwara Roa (CSE)**, Head of the ***Department for Computer Science Engineering*** for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

BATHULA NEHA 2520090013

CHAKRIKA REDDY 2520090044

HARSHIKA 2520090047

Abstract

Linux supports multitasking, and shell pipelines let the output of one command become the input of the next. When commands are executed one by one, extra steps are needed to pass data between them, and intermediate files increase manual work and file-handling overhead. To address this, our project focuses on the design and development of a Multi-Process Command Pipeline System that recreates command pipelines in C using Linux system calls.

The program reads a line such as “ls | grep .c | wc -l” at run time and splits it on the pipe symbol. It creates one pipe between each pair of adjacent commands and forks one child process per command. Each child redirects its standard input and output with dup2() and runs its command with execvp(), while the parent waits for every child with waitpid(). The system supports up to 10 commands per pipeline, handles pipe, fork and exec failures, and closes unused file descriptors.

In summary, the project demonstrates concurrent process execution and inter-process communication (IPC) without temporary files, and builds practical knowledge of process management, pipes, descriptor redirection and parent–child synchronization in Linux system programming.

Index

S. No.	Chapter	Topics / Subtopics	Page No.
	Acknowledgement		i
	Abstract		ii
1	Introduction		8–9
		1.1 Background of the Project	
		1.2 Problem Statement	
		1.3 Scope of the Project	
		1.4 Objectives of the Project	
		1.5 Importance of the Application	
		1.6 Target Users / Audience	
2	OS Concepts and Background Study		10–10
		2.1 Operating System Concepts Related to the Project	
		2.2 Existing Methods / Algorithms	
		2.3 Proposed Approach	
		2.4 Advantages of the Proposed Approach	
3	System Requirements		11–11
		3.1 Hardware Requirements	
		3.2 Software Requirements	
		3.3 Operating System Environment – Ubuntu / WSL	
		3.4 Development Tools and Compilers	
4	Technology and Tools Used		12–13
		4.1 C / C++ Programming	
		4.2 GCC Compiler	
		4.3 Linux System Calls	
		4.4 POSIX Threads / Synchronization	
		4.5 Linux Commands and Utilities	
		4.6 Other Tools / Libraries Used	
5	System Architecture and Design		14–15
		5.1 Overall System Architecture	
		5.2System Workflow	
		5.3 Flowchart / Data Flow	
		5.4 Module Description	

		5.5 Input and Output Design	
6	**Implementation**		**16–18**
		6.1 Module-wise Implementation	
		6.2 Algorithm / Pseudocode	
		6.3 Core C / C++ Implementation	
		6.4 Linux System Calls / APIs Used	
		6.5 Process / Thread Implementation	
		6.6 IPC / Synchronization Implementation	
		6.7 Memory Management Implementation	
		6.8 Optional User Interface Implementation	
7	**Execution and Demonstration**		**19–20**
		7.1 Compilation in Ubuntu	
		7.2 Execution Commands	
		7.3 Sample Inputs and Outputs	
		7.4 Screenshots of Execution	
		7.5 Demonstration of OS Concepts	
8	**Conclusion**		**21–21**
		8.1 Summary of the Project	
		8.2 Objectives Achieved	
		8.3 Results Achieved	
		8.4 Skills and Learning Outcomes	
9	**References**		**22–22**
		9.1 Books	
		9.2 Research Papers	
		9.3 Linux / GNU Documentation	
		9.4 Online Resources	

CHAPTER 1
INTRODUCTION

1.1 Background of the Project

Linux supports multitasking, so many processes can run at the same time. A command pipeline connects commands so that the output of one command becomes the input of the next. For example, in “ls | grep .c | wc -l”, the command ls lists the files, grep .c keeps the lines matching .c, and wc -l counts the lines. Our project builds this idea in C using Linux system calls, to show how pipelines work internally.

1.2 Problem Statement

When commands are executed one by one, extra steps are needed to pass data between them. Intermediate files may be used to store the output, which increases manual work and file-handling overhead. The problems identified are:

1. It is difficult to connect multiple commands programmatically.
2. Processes need efficient communication between them.
3. Temporary files are used unnecessarily.
4. Managing multiple child processes is complex.
5. Proper error handling and synchronization are needed.

Proposed solution: a C-based system that accepts commands separated by the pipe symbol (|) and runs them concurrently using child processes and POSIX pipes, in the form cmd1 | cmd2 | cmd3.

1.3 Scope of the Project

- A command-line program that accepts a command pipeline typed at run time.
- Support for up to 10 commands in one pipeline, each with arguments split on spaces and tabs.
- One child process per command, connected by pipes, with the parent waiting for all children.
- Not in scope (current limitations): shell operators such as &&, ||, > or <; quoted-string argument parsing; wildcard expansion. Input is accepted once per program run.

1.4 Objectives of the Project

The primary objective is to implement a Linux command pipeline execution system using C. The specific objectives are to:

1. Accept multiple commands from the user.
2. Separate commands using the | symbol.
3. Create child processes using fork().
4. Establish communication using pipe().
5. Redirect input and output using dup2().
6. Execute commands using execvp().

7. Support a maximum of 10 commands.
8. Handle invalid commands and system-call failures.
9. Synchronize processes using waitpid().
10. Display the final command output.

Expected outcome: a working command-line application that demonstrates concurrent process execution and inter-process communication.

1.5 Importance of the Application

- Demonstrates Linux process management.
- Improves understanding of IPC mechanisms.
- Shows concurrent command execution.
- Reduces dependence on temporary files.
- Demonstrates POSIX system calls.
- Develops practical operating-system programming skills.

1.6 Target Users / Audience

- Operating-system and system-programming students.
- Learners of Linux shell internals and shell development.
- Anyone exploring log-file analysis, data processing (filtering, sorting and counting) and basic automated command-line workflows built on command chaining.

CHAPTER 2
OS CONCEPTS AND BACKGROUND STUDY

2.1 Operating System Concepts Related to the Project

Concept	System call / mechanism	Role in the project
Process creation	`fork()`	Creates a new child process for each command
Inter-process communication	`pipe()`	Creates a one-way communication channel between adjacent commands
File descriptors & redirection	`dup2()`	Redirects standard input and output to pipe ends
Program loading	`execvp()`	Replaces the child with the requested Linux command
Process synchronization	`waitpid()`	Parent waits for each child to end
Resource management	`close()`	Closes unused file descriptors

2.2 Existing Methods / Algorithms

In the existing approach, commands are run separately and the output of each is stored in an intermediate file before the next command reads it:

```
$ ls > files.txt
$ grep .c files.txt > result.txt
$ wc -l < result.txt
```

The limitations of this approach are that it requires intermediate files, involves extra file operations, needs more manual steps and makes command chaining less convenient. Linux shells already support pipelines; our project recreates this using C system calls to show how pipelines work internally.

2.3 Proposed Approach

The proposed system is a C program that reads the whole pipeline at once, splits it into commands, and creates N−1 pipes for N commands. It forks one child per command, wires each child's standard input and output to the neighbouring pipes, and lets the commands run concurrently. The parent finally waits for every child to complete.

2.4 Advantages of the Proposed Approach

- Direct communication between processes.
- No intermediate files required.
- Concurrent command execution.
- Dynamic command input at run time.
- Efficient data streaming.
- Basic error detection.

CHAPTER 3
SYSTEM REQUIREMENTS

3.1 Hardware Requirements

The project has modest requirements; the following are typical minimums for running Ubuntu or WSL and compiling C programs:

Component	Requirement
Processor	Any modern x86-64 / ARM processor
Memory (RAM)	1 GB or more (4 GB recommended)
Storage	A few hundred MB of free disk space
Peripherals	Keyboard and display for the terminal

3.2 Software Requirements

Software	Purpose
Ubuntu Linux (or WSL)	Operating environment
GCC compiler	Compiling pipeline.c
Linux terminal	Execution and testing
Gedit / VS Code	Code development

3.3 Operating System Environment – Ubuntu / WSL

The program uses POSIX system calls such as fork(), pipe(), dup2(), execvp() and waitpid(), so it must run on Linux. Ubuntu is the primary environment; Windows users can use the Windows Subsystem for Linux (WSL).

3.4 Development Tools and Compilers

The source code is written in Gedit or VS Code and compiled with GCC:

```
$ gcc pipeline.c -o pipeline
```

CHAPTER 4
TECHNOLOGY AND TOOLS USED

Technology	Purpose
C Programming	Main programming language
Ubuntu Linux	Operating environment
GCC Compiler	Compilation
POSIX System Calls	Process management
Linux Pipes	Inter-process communication
Linux Terminal	Execution and testing
Gedit / VS Code	Code development

4.1 C / C++ Programming

The project is written in C, which gives direct access to Linux system calls and is the standard language for operating-system programming.

4.2 GCC Compiler

GCC compiles pipeline.c into the executable pipeline using “gcc pipeline.c -o pipeline”.

4.3 Linux System Calls

System call	Purpose
`fork()`	Creates a new child process
`pipe()`	Creates a one-way communication channel
`dup2()`	Redirects standard input and output
`execvp()`	Replaces the child with a Linux command
`waitpid()`	Waits for a child process to end
`close()`	Closes unused file descriptors

4.4 POSIX Threads / Synchronization

This project uses multiple processes rather than threads. Synchronization is achieved by the parent process calling waitpid() on every child; WIFEXITED() and WEXITSTATUS() are used to check whether each child ended normally and with what exit status.

4.5 Linux Commands and Utilities

The commands used for testing pipelines include ls, grep, wc, sort, pwd and printf, together with gcc and the Linux terminal.

4.6 Other Tools / Libraries Used

Function	Purpose
`fgets()`	Reads user input
`strtok_r()`	Separates commands and arguments
`perror()`	Displays system errors
`WIFEXITED()`	Checks normal process termination
`WEXITSTATUS()`	Gets the process exit status

CHAPTER 5
SYSTEM ARCHITECTURE AND DESIGN

5.1 Overall System Architecture

The user types a pipeline, which is read with fgets() and split with strtok_r(). The parent process then creates the pipes and forks one child per command. Each child reads from the previous pipe (or the terminal for the first command) and writes to the next pipe (or the terminal for the last command).

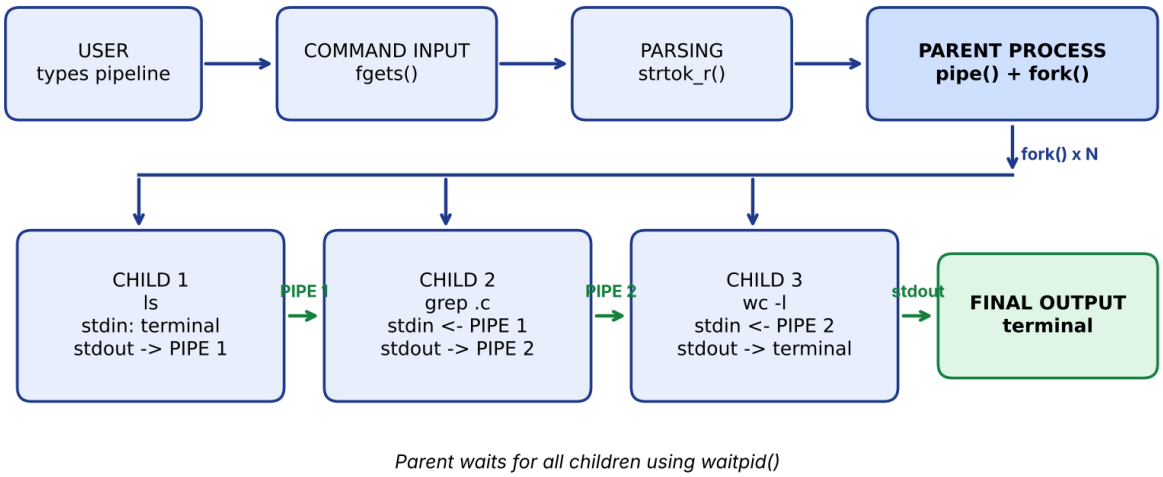


Figure 5.1: Architecture of the Multi-Process Command Pipeline System (example: ls | grep .c | wc -l)

5.2 System Workflow

Step	Stage	Description
1	User input	A prompt is shown and the user types: ls | grep .c | wc -l
2	Command parsing	The input is split into 3 commands: ls, grep .c, wc -l
3	Pipe creation	3 commands need 2 pipes (one fewer than the commands)
4	Process creation	The parent creates 3 child processes using fork()
5	I/O redirection	Processes are connected using dup2()
6	Command execution	Each child runs its command using execvp()
7	Output generation	The final process displays the result
8	Completion	The parent waits for all children using waitpid()

Number of pipes = Number of commands − 1 (for example, 3 commands → 2 pipes).

5.3 Flowchart / Data Flow

1. Start the program and display the prompt.
2. Read the input line with fgets(); if it is empty, print “Error: No commands entered.” and stop.
3. Split the line on | into commands (maximum 10) and split each command into arguments.
4. For each command: create a pipe (except for the last command), then fork().
5. In the child: redirect stdin from the previous pipe and stdout to the new pipe using dup2(), close unused descriptors, and call execvp().
6. In the parent: close the pipe ends that are no longer needed and remember the read end for the next command.
7. After all children are launched, call waitpid() for each child and check its exit status.
8. Display the final output and end the program.

Data flow: terminal → child 1 → pipe 1 → child 2 → pipe 2 → … → child N → terminal.

5.4 Module Description

Module	Responsibility
User Input Module	Accepts commands separated by |
Command Parser	Separates commands and arguments
Process Manager	Creates child processes using fork()
Pipe Manager	Creates communication channels using pipe()
Execution Module	Runs commands using execvp()
Synchronization Module	Waits for children using waitpid()

5.5 Input and Output Design

- **Input:** one line of up to 1024 characters (MAX_INPUT), containing commands separated by |, each with up to 20 arguments (MAX_ARGS), and up to 10 commands (MAX_COMMANDS).
- **Output:** the output of the last command in the pipeline is displayed in the terminal.
- **Errors:** reported with perror() or messages such as “Error: No commands entered.” for empty input.

CHAPTER 6
IMPLEMENTATION

6.1 Module-wise Implementation

Phase	Work done	Key call
1. Input handling	Reads up to 1024 characters, including the null character	`fgets()`
2. Command parsing	Splits on |; arguments split on spaces and tabs	`strtok_r()`
3. Pipe creation	One pipe for every command except the last	`pipe()`
4. Process creation	Each command runs in its own child process	`fork()`
5. Communication	Previous pipe to stdin, new pipe to stdout	`dup2()`
6. Command execution	The child replaces itself with the Linux command	`execvp()`
7. Synchronization	The parent waits for every launched child	`waitpid()`

Features implemented: dynamic command input; multi-process execution; pipe-based communication; support for up to 10 commands; I/O redirection using dup2(); error handling (empty input, pipe/fork/exec failure, unsuccessful child exit); process synchronization; and resource management by closing unused pipe descriptors.

Current limitations:

- No shell operators such as &&, ||, > or <.
- No quoted-string argument parsing.
- No wildcard expansion.
- Maximum of 10 commands.
- Input is accepted once per program run.
- Some malformed pipelines are not fully detected.

6.2 Algorithm / Pseudocode

```
read input line
if input is empty: print "Error: No commands entered."; exit
split input on '|' into commands[0..count-1]      (count <= 10)
previous_read = -1
for i = 0 .. count-1:
    if i < count-1: create pipe fd[2]             (on failure: perror, stop)
    pid = fork()
    if pid == 0:                                  /* child */
        if previous_read != -1: dup2(previous_read, STDIN_FILENO)
        if i < count-1:         dup2(fd[1], STDOUT_FILENO)
        close all unused pipe descriptors
        execvp(argv[i][0], argv[i]); perror on failure; exit(1)
    else:                                         /* parent */
        close previous_read and fd[1]; previous_read = fd[0]
for each launched child: waitpid(pids[i], &status, 0)
    check WIFEXITED(status) and WEXITSTATUS(status)
```

6.3 Core C / C++ Implementation

The following are simplified extracts of the core code. The full program also includes error checking and descriptor cleanup.

(a) Project configuration and reading input

```
#define MAX_COMMANDS 10
#define MAX_ARGS 20
#define MAX_INPUT 1024

fgets(input, sizeof(input), stdin);
input[strcspn(input, "\n")] = '\0';
```

(b) Separating commands

```
char *token = strtok_r(input, "|", &saveptr);

while (token != NULL && count < MAX_COMMANDS) {
    while (*token == ' ' || *token == '\t')
        token++;

    commands[count++] = token;
    token = strtok_r(NULL, "|", &saveptr);
}
```

(c) Creating pipes and processes

```
if (i < count - 1 && pipe(fd) == -1) {
    perror("pipe");
    failed = 1;
    break;
}

pid_t pid = fork();
```

(d) Redirecting input and output, executing, and waiting

```
if (previous_read != -1)
    dup2(previous_read, STDIN_FILENO);

if (i < count - 1)
    dup2(fd[1], STDOUT_FILENO);

execvp(argv[i][0], argv[i]);

waitpid(pids[i], &status, 0);
```

6.4 Linux System Calls / APIs Used

The system calls used are fork(), pipe(), dup2(), execvp(), waitpid() and close(); the supporting library functions are fgets(), strtok_r(), perror(), WIFEXITED() and WEXITSTATUS() (see Chapter 4).

6.5 Process / Thread Implementation

The program consists of one parent process and one child process per command. The children run concurrently, and the parent records their process IDs in the pids[] array so that it can wait for each of them later. No threads are used.

6.6 IPC / Synchronization Implementation

Anonymous pipes created with pipe() carry data from the standard output of one process to the standard input of the next. The parent synchronizes with the children using waitpid(), so it only finishes after every launched child has ended, and it inspects each exit status with WIFEXITED() and WEXITSTATUS().

6.7 Memory Management Implementation

The program uses fixed-size buffers whose sizes come from MAX_INPUT, MAX_ARGS and MAX_COMMANDS. Resource management mainly concerns file descriptors: unused pipe ends are closed in both the parent and the children so that readers see end-of-file correctly. Supporting an unlimited number of commands through dynamic memory is listed as future work.

6.8 Optional User Interface Implementation

The interface is a simple text prompt in the terminal:

```
Multi-Process Command Pipeline System
Enter commands separated by | :
>
```

CHAPTER 7
EXECUTION AND DEMONSTRATION

7.1 Compilation in Ubuntu

Save the source as pipeline.c and compile it with GCC:

```
$ gcc pipeline.c -o pipeline
```

7.2 Execution Commands

```
$ ./pipeline
Multi-Process Command Pipeline System
Enter commands separated by | :
> ls | grep .c | wc -l
```

7.3 Sample Inputs and Outputs

Example output (not a verified test result). Assumed directory contents: main.c, test.c, notes.txt, data.txt.

```
user@linux:~/project$ gcc pipeline.c -o pipeline
user@linux:~/project$ ./pipeline
Multi-Process Command Pipeline System
Enter commands separated by | :
> ls | grep .c | wc -l
2
```

Explanation: ls lists the directory entries; grep .c keeps the lines matching its pattern; wc -l counts the filtered lines; the final result is shown in the terminal.

Test plan. The strategy is to run different command pipelines and compare the output with what is expected. The Actual Result and Status columns remain “Not yet verified” until the program is run and the real outputs are recorded.

No.	Test case	Input	Expected result	Actual	Status
1	Single command	`pwd`	Displays the current working directory	Not yet verified	Not yet verified
2	Two commands	`ls | wc -l`	Displays the number of lines produced by ls	Not yet verified	Not yet verified
3	Three commands	`ls | grep .c | wc -l`	Displays the count of directory-listing lines matching .c	Not yet verified	Not yet verified
4	Sorting	`printf banana | sort`	Displays banana	Not yet verified	Not yet verified
5	Invalid command	`invalidcommand`	Displays a command execution error and a pipeline error message	Not yet verified	Not yet verified
6	Empty input	`(press Enter)`	Error: No commands entered.	Not yet verified	Not yet verified

7.4 Screenshots of Execution

[Insert screenshots of the compilation step and of each test case here after running the program.]

7.5 Demonstration of OS Concepts

The execution demonstrates the following operating-system concepts:

- Multiple child processes are created (fork()).
- Pipes connect adjacent processes (pipe()).
- Commands run concurrently.
- Intermediate output moves through pipes instead of files.
- The final process displays the result, and the parent waits for all children (waitpid()).

CHAPTER 8
CONCLUSION

8.1 Summary of the Project

The Multi-Process Command Pipeline System is a C-based Linux application that runs multiple commands concurrently using pipes. It uses fork(), pipe(), dup2(), execvp() and waitpid(), and builds practical knowledge of process management, inter-process communication and Linux system programming.

8.2 Objectives Achieved

- Accepts commands dynamically at run time, separated by |.
- Creates a separate child process for each command (up to 10) and connects them with pipes.
- Redirects input and output using dup2() and executes commands using execvp().
- Handles empty input, pipe/fork/exec failures and unsuccessful child exits.
- Synchronizes the parent with all children using waitpid(), and closes unused descriptors.

8.3 Results Achieved

The system streams data directly between processes without intermediate files, and for the example pipeline ls | grep .c | wc -l it is expected to display the count of matching entries. Final measured results are to be recorded in the test table in Section 7.3 once the program has been run.

Applications and impact: Linux shell development, log-file analysis, data processing (filtering, sorting and counting), system administration basics and OS education.

8.4 Skills and Learning Outcomes

Through this project we learned how processes are created, how pipes connect them, how file descriptors are redirected, and how the parent manages its children.

Future enhancements:

1. Build a complete custom Linux shell.
2. Add input and output redirection.
3. Support quoted arguments and special characters.
4. Add background process execution.
5. Improve command parsing and validation.
6. Support unlimited commands with dynamic memory.
7. Add execution-time measurement.
8. Improve error reporting and process monitoring.

CHAPTER 9
REFERENCES

9.1 Books

1. A. Silberschatz, P. B. Galvin, G. Gagne, Operating System Concepts, Wiley.
2. W. R. Stevens, S. A. Rago, Advanced Programming in the UNIX Environment, Addison-Wesley.
3. M. Kerrisk, The Linux Programming Interface, No Starch Press.

9.2 Research Papers

[Add any research papers referred to, or remove this subsection.]

9.3 Linux / GNU Documentation

1. Linux manual pages: fork(2), pipe(2), dup2(2), execvp(3), waitpid(2).
2. GNU C Library Reference Manual.
3. GCC, the GNU Compiler Collection – online documentation.

9.4 Online Resources

[Add the websites or tutorials used.]